Distributed System Fundamentals

many machines that must act like one

Watch the 4-min video

In one line: a distributed system is several computers that cooperate over a network to look like one; you build one to scale, survive failures and get close to users, and you pay with partial failures, unreliable networks, unsynchronised clocks and harder consistency.

1 From one box to many

Your app runs on one server with one database. It's simple: one copy of the data, one clock, and a call either works or the whole thing crashes. Then the database fills up, one crash takes everything down, and users on another continent wait a second for every page. You add machines, and the app becomes a **distributed system**: several computers, connected only by a network, that must behave like one system for the user.

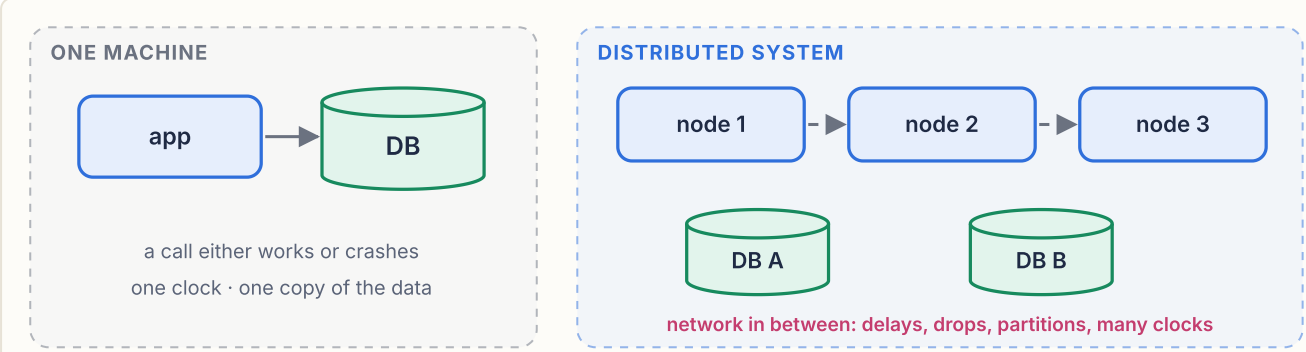


Figure 1. On one machine, failure is all or nothing. Across machines, the network sits between every pair of parts.

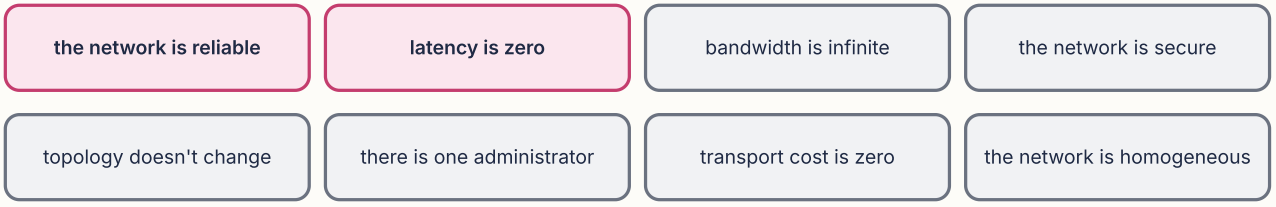
2 Why distribute at all

- **Scale:** one machine has a ceiling on CPU, memory, disk and connections. Many machines don't.
- **Availability:** if one machine dies, others keep serving. A single box is a single point of failure.
- **Latency:** put copies of the data near users, in several regions.
- **Organisation:** separate services let separate teams deploy independently.

3 What gets harder

Four new problems arrive the moment there are two machines, and the rest of Phase 4 deals with them one by one:

- **Partial failure** (lesson 2): one part fails while the rest keeps running, and you often can't tell a slow node from a dead one.
- **Unreliable network** (lesson 3): messages are delayed, lost, duplicated or reordered, and links can split the cluster in two.
- **No shared clock** (lessons 5 to 7): each machine's clock drifts, so timestamps can't safely order events.
- **Agreement** (lessons 8 to 13): several nodes must agree on a leader, a value or a lock, even while some of them fail.



the eight fallacies of distributed computing (Deutsch, Gosling): all false, all assumed by new code

Figure 2. The eight fallacies: assumptions that are true on one machine and false across a network. The first two cause most outages.

4 Consistency: what can a reader see?

With several copies of the data, a reader might see an old copy. Systems choose a point on a scale: **strong consistency** (every read sees the latest write, at the cost of coordination and latency) or **eventual consistency** (copies converge if writes stop, and reads may be briefly stale). The CAP theorem says that during a network **partition** you must pick consistency or availability; PACELC adds that even without a partition, you trade latency for consistency.

WORKED EXAMPLE: FROM ONE DATABASE TO A REPLICATED CLUSTER

A monolith's single Postgres becomes a cluster: one **primary** in zone A takes all writes, and **replicas** in zones B and C copy its changes and serve reads. Now the app survives the loss of a zone and spreads read load. But it inherits new questions: replicas **lag** behind, so a user may not see their own update; someone must detect a dead primary and **promote** a replica; and if the old primary was only cut off, two nodes may both accept writes (split-brain, lesson 12).

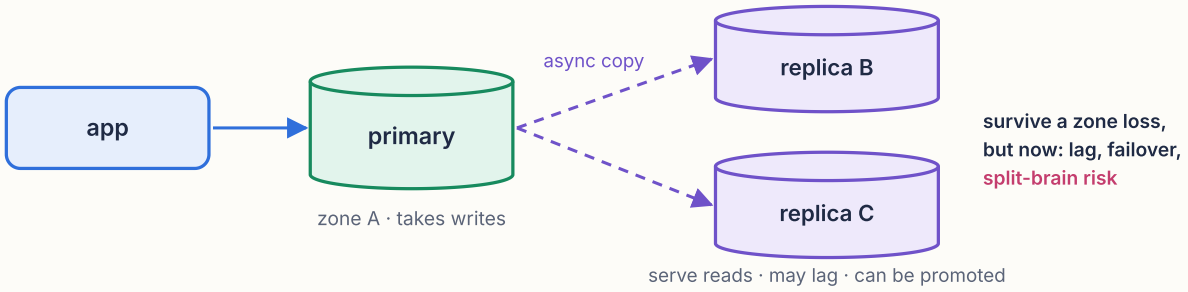


Figure 3. Replication buys availability and read scale, and brings lag, failover and split-brain with it.

5 How to think about a distributed design

For every arrow in your diagram, ask three questions: what if this call is **slow**, what if it **fails**, and what if it **succeeds twice**? Timeouts, retries with backoff, idempotency and replication are the everyday answers. Distribute only when you need to: a well-tuned single database goes a long way.

COMMON MISTAKES

- Treating a remote call like a local function call: no timeout, no retry plan.
- Ordering events across machines by their wall-clock timestamps.
- Splitting into microservices before the single system is actually a bottleneck.
- Assuming replicas are always up to date right after a write.

6 Key terms

Distributed system: several computers cooperating over a network to act as one

Node: one machine or process in the system

Partial failure: some parts fail while others keep working

Replication: keeping copies of data on several nodes

Partition: a network split: nodes are up but can't reach each other

Strong consistency: every read sees the latest write

Eventual consistency: copies converge over time; reads may be stale

CAP / PACELC: consistency vs availability under partition; latency vs consistency otherwise

7 Interview questions

Q1. What is a distributed system, and why build one? BASIC

Several computers that cooperate over a network and appear as one system. You build one to scale past one machine, to stay available when a machine fails, and to serve users from nearby regions.

Q2. What new problems appear once you have more than one machine? BASIC

Partial failures, an unreliable network (delays, loss, partitions), no shared clock to order events, and the need for nodes to agree on things like a leader or the latest value.

Q3. Your single Postgres is at its limit. What do you do before going distributed? SCENARIO

Measure first: add indexes, fix slow queries, add a cache and read replicas, scale the machine up. Shard or split only when one primary truly can't take the write load.

Q4. You add read replicas and a user doesn't see the profile change they just saved. Why, and how do you fix it? SCENARIO

Replication lag: the read hit a replica that hadn't applied the write yet. Read your own writes from the primary for a short window, or route the user to a replica that has caught up to their write position.

Q5. Is a system with one app server and one managed database distributed? TRICKY

Yes, in the sense that matters: the app and database talk over a network, so calls can time out, fail half-way or be retried, and you must handle that. Managed services hide some, not all, of it.

Q6. What does CAP really force you to choose? TRICKY

Only during a partition: either reject some requests to stay consistent, or answer from both sides and risk divergence. When the network is fine you can have both, but you still trade latency for consistency (PACELC).

Next: SD-H-002 · Partial Failures backendroadmap.com